Lab Assignment 8

1. (Username Validator – Apply AI in Authentication Context)

Task:

Use AI to generate at least 3 assert test cases for a function is_valid_username(username) and then implement the function using Test-Driven Development principles.

Requirements:

- Username length must be between 5 and 15 characters.
- Must contain only alphabets and digits.
- Must not start with a digit.
- No spaces allowed.

Example Assert Test Cases:

- assert is_valid_username("User123") == True
- assert is_valid_username("12User") == False
- assert is_valid_username("Us er") == False

Expected Output:

- Username validation logic successfully passing all AI-generated test cases.

min_stack.py 1.py

Sem-VI > AI-Assisted > Ass_8 > 1.py > ...

```python
def is_valid_username(username):
    if not 15 > len(username) > 5:
        return False
    if not username.isalnum():
        return False
    if not username[0].isalpha():
        return False
    if " " in username:
        return False
    return True

assert is_valid_username("User123") == True
assert is_valid_username("12User") == False
assert is_valid_username("Us er") == False
```

Output:

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\rhars\Documents\.Dev\Uni> & C:/Python314/python.exe c:/Users/rhars/Documents/.Dev/Uni/Sem-VI/AI-Assisted/Ass_8/1.py
PS C:\Users\rhars\Documents\.Dev\Uni>
```

2. (Even–Odd & Type Classification – Apply AI for Robust Input Handling)

Task:

Use AI to generate at least 3 assert test cases for a function classify_value(x) and implement it using conditional logic and loops.

Requirements:

- If input is an integer, classify as "Even" or "Odd".
- If input is 0, return "Zero".
- If input is non-numeric, return "Invalid Input".

Example Assert Test Cases:

- assert classify_value(8) == "Even"

- assert classify_value(7) == "Odd"
- assert classify_value("abc") == "Invalid Input"

Expected Output:

- Function correctly classifying values and passing all test cases.

min_stack.py | r_1.py | r_2.py | r_3.py | 1.py | 2.py

Sem-VI > AI-Assisted > Ass_8 > 2.py > ...

```python
def classify_value(x):
    try:
        if x%2 == 0:
            return "Even"
        elif x%2 == 1:
            return "Odd"
        elif x == 0:
            return "Zero"
    except TypeError:
        return "Invalid Input"

assert classify_value(8) == "Even"
assert classify_value(7) == "Odd"
assert classify_value("abc") == "Invalid Input"
```

Output:

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\rhars\Documents\.Dev\Uni> & C:/Python314/python.exe c:/Users/rhars/Documents/.Dev/Uni/Sem-VI/AI-Assisted/Ass_8/2.py
PS C:\Users\rhars\Documents\.Dev\Uni>
```

3. (Palindrome Checker – Apply AI for String Normalization)

Task:

Use AI to generate at least 3 assert test cases for a function is_palindrome(text) and implement the function.

Requirements:

- Ignore case, spaces, and punctuation.
- Handle edge cases such as empty strings and single characters.

Example Assert Test Cases:

- assert is_palindrome("Madam") == True
- assert is_palindrome("A man a plan a canal Panama") == True
- assert is_palindrome("Python") == False

Expected Output #3:

- Function correctly identifying palindromes and passing all AI-generated tests.

3.py × Release Notes

Sem-VI > AI-Assisted > Ass_8 > 3.py > ...

```python
def is_palindrome(text):
    # Ignore case, spaces, and punctuation.
    text = text.lower()
    text = list[Any](text)
    j = len(text)-1
    i = 0
    while i < j:
        if text[i] == " ":
            i += 1
        if text[j] == " ":
            j -= 1
        if text[i] != text[j]:
            return False
        i += 1
        j -= 1
    return True

assert is_palindrome("Madam") == True
assert is_palindrome("A man a plan a canal Panama") == True
assert is_palindrome("Python") == False
```

Problems Output Debug Console Terminal Python

```
PS C:\Users\rhars\Documents\.Dev\Uni> & C:/Python314/python.exe c:/Users/rhars/Documents/.Dev/Uni/Sem-VI/AI-Assisted/Ass_8/3.py
PS C:\Users\rhars\Documents\.Dev\Uni>
```

4. (Email ID Validation – Apply AI for Data Validation)

Task:

Use AI to generate at least 3 assert test cases for a function validate_email(email) and implement the function.

Requirements:

- Must contain @ and .
- Must not start or end with special characters.
- Should handle invalid formats gracefully.

Example Assert Test Cases:

- assert validate_email("user@example.com") == True
- assert validate_email("userexample.com") == False
- assert validate_email("@gmail.com") == False

Expected Output:

- Email validation function passing all AI-generated test cases and handling edge cases correctly.

```python
def validate_email(email):
    if not "@" in email or not "." in email:
        return False
    if not email[0].isalpha() or not email[-1].isalpha():
        return False
    at_index = email.find("@")
    last_dot_index = email.rfind(".")

    if at_index == -1 or last_dot_index == -1:
        return False
    if at_index >= last_dot_index - 1:
        return False

    if email.count("@") != 1:
        return False

    return True

assert validate_email("user@example.com") == True
assert validate_email("userexample.com") == False
assert validate_email("@gmail.com") == False

# Edge Case
assert validate_email("") == False
```

Output:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\Himes\Documents\.Dev\Python> & C:/Users/Himes/AppData/Local/Programs/Python/Python314/python.exe c:/Users/Himes/Documents/.Dev/Python/ass_8_4.py
PS C:\Users\Himes\Documents\.Dev\Python>
```

5. (Perfect Number Checker – Test Case Design)

Function:

Check if a number is a perfect number (sum of divisors = number).

Test Cases to Design:

- Normal case: 6 → True, 10 → False.
- Edge case: 1.
- Negative number case.
- Larger case: 28.

Requirement:

Validate correctness with assertions.

```python
def is_perfect(n):
    if n <= 1:
        return False
    divisors_sum = 0
    for i in range(1, n):
        if n % i == 0:
            divisors_sum += i
    return divisors_sum == n

# Normal Case
assert is_perfect(6) == True
assert is_perfect(10) == True

# Edge Case
assert is_perfect(1) == False

# Negative Case
assert is_perfect(10) == False

# Large Number Case
assert is_perfect(28) == True
```

Output:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\Himes\Documents\.Dev\Python> & C:/Users/Himes/AppData/Local/Programs/Python/Python314/python.exe c:/Users/Himes/Documents/.Dev/Python/ass_8_5.py
PS C:\Users\Himes\Documents\.Dev\Python>
```

6. (Abundant Number Checker – Test Case Design)

Function:

Check if a number is abundant (sum of divisors > number).

Test Cases to Design:

- Normal case: 12 → True, 15 → False.
- Edge case: 1.
- Negative number case.
- Large case: 945.

Requirement:

Validate correctness with unittest

```python
import unittest

def is_abundant(n):
    if not isinstance(n, int) or n <= 0:
        return False

    divisors_sum = 0
    for i in range(1, n):
        if n % i == 0:
            divisors_sum += i

    return divisors_sum > n


class TestAbundantNumber(unittest.TestCase):

    def test_normal_cases(self):
        self.assertTrue(is_abundant(12))
        self.assertFalse(is_abundant(15))

    def test_edge_case(self):
        self.assertFalse(is_abundant(1))

    def test_negative_numbers(self):
        self.assertFalse(is_abundant(-12))
        self.assertFalse(is_abundant(-1))

    def test_large_case(self):
        self.assertTrue(is_abundant(945))

    def test_zero(self):
        self.assertFalse(is_abundant(0))


if __name__ == '__main__':
    unittest.main()
```

Output:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\Himes\Documents\.Dev\Python> & C:/Users/Himes/AppData/Local/Programs/Python/Python314/python.exe c:/Users/Himes/Documents/.Dev/Python/ass_8_6.py
.....
----------------------------------------------------------------------
Ran 5 tests in 0.001s

OK
PS C:\Users\Himes\Documents\.Dev\Python>
```

7. (Deficient Number Checker – Test Case Design)

Function:

Check if a number is deficient (sum of divisors < number).

Test Cases to Design:

- Normal case: 8 → True, 12 → False.
- Edge case: 1.
- Negative number case.
- Large case: 546.

Requirement:

Validate correctness with pytest.

ass_8_4.py | ass_8_5.py | ass_8_6.py | ass_8_7_1.py | ass_8_7_2.py

ass_8_7_1.py > is_deficient

```python
def is_deficient(n):
    # Basic input validation
    if not isinstance(n, int) or n <= 0:
        return False

    # Find sum of proper divisors
    divisors_sum = 0
    for i in range(1, n):
        if n % i == 0:
            divisors_sum += i

    return divisors_sum < n
```

ass_8_4.py | ass_8_5.py | ass_8_6.py | ass_8_7_1.py | ass_8_7_2.py

ass_8_7_2.py > ...

```python
from ass_8_7_1 import is_deficient

def test_normal_cases():
    assert is_deficient(8) == True, "8 should be deficient (1+2+4=7 < 8)"
    assert is_deficient(12) == False, "12 should NOT be deficient (1+2+3+4+6=16 > 12)"

def test_edge_case_one():
    assert is_deficient(1) == True, "1 should be deficient (sum=0 < 1)"

def test_negative_numbers():
    assert is_deficient(-8) == False, "Negative numbers should return False"
    assert is_deficient(-1) == False, "Negative numbers should return False"

def test_large_case():
    assert is_deficient(546) == True, "546 should be deficient"

def test_zero():
    assert is_deficient(0) == False, "Zero should return False"

def test_perfect_number():
    assert is_deficient(6) == False, "6 is perfect, should NOT be deficient"

def test_prime_number():
    assert is_deficient(7) == True, "7 is prime, should be deficient"

def test_additional_cases():
    assert is_deficient(2) == True, "2 should be deficient (1 < 2)"
    assert is_deficient(10) == True, "10 should be deficient (1+2+5=8 < 10)"
    assert is_deficient(15) == True, "15 should be deficient (1+3+5=9 < 15)"
    assert is_deficient(28) == False, "28 is perfect, should NOT be deficient"
```

Output:

PROBLEMS | OUTPUT | DEBUG CONSOLE | TERMINAL | PORTS

```
PS C:\Users\Himes\Documents\.Dev\Python> pytest ass_8_7_1.py -v
========================================================================================== test session starts ==========================================================================================
platform win32 -- Python 3.14.0, pytest-9.0.2, pluggy-1.6.0 -- C:\Users\Himes\AppData\Local\Programs\Python\Python314\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\Himes\Documents\.Dev\Python
collected 0 items

========================================================================================== no tests ran in 0.01s ==========================================================================================
PS C:\Users\Himes\Documents\.Dev\Python>
```

8. Write a function LeapYearChecker and validate its implementation using 10 pytest test cases.

ass_8_4.py | ass_8_5.py | ass_8_6.py | ass_8_7_1.py | ass_8_7_2.py | ass_8_8_1.py | ass_8_8_2.py

ass_8_8_1.py > is_leap_year

```python
def is_leap_year(year):
    if not isinstance(year, int):
        return False
    if year <= 0:
        return False
    if year % 400 == 0:
        return True
    if year % 100 == 0:
        return False
    if year % 4 == 0:
        return True
    return False
```

```python
from ass_8_8_1 import is_leap_year

def test_divisible_by_400():
    assert is_leap_year(2000) == True
    assert is_leap_year(1600) == True
    assert is_leap_year(2400) == True

def test_divisible_by_100_but_not_400():
    assert is_leap_year(1900) == False
    assert is_leap_year(2100) == False
    assert is_leap_year(2200) == False

def test_divisible_by_4_but_not_100():
    assert is_leap_year(2004) == True
    assert is_leap_year(2008) == True
    assert is_leap_year(2012) == True

def test_not_divisible_by_4():
    assert is_leap_year(2001) == False
    assert is_leap_year(2002) == False
    assert is_leap_year(2003) == False

def test_edge_case_year_0():
    assert is_leap_year(0) == False

def test_negative_years():
    assert is_leap_year(-400) == False
    assert is_leap_year(-2024) == False

def test_recent_years():
    assert is_leap_year(2020) == True
    assert is_leap_year(2021) == False
    assert is_leap_year(2024) == True

def test_century_boundaries():
    assert is_leap_year(1899) == False
    assert is_leap_year(1901) == False
    assert is_leap_year(2399) == False

def test_future_years():
    assert is_leap_year(3000) == False
    assert is_leap_year(3200) == False
    assert is_leap_year(4000) == True

def test_invalid_inputs():
    assert is_leap_year("2024") == False
    assert is_leap_year(2024.5) == False
    assert is_leap_year(None) == False
```

Output:

```
PS C:\Users\Himes\Documents\.Dev\Python> pytest ass_8_8_2.py -v
========================================== test session starts ==========================================
platform win32 -- Python 3.14.0, pytest-9.0.2, pluggy-1.6.0 -- C:\Users\Himes\AppData\Local\Programs\Python\Python314\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\Himes\Documents\.Dev\Python
collected 10 items

ass_8_8_2.py::test_divisible_by_400 PASSED                                                    [ 10%]
ass_8_8_2.py::test_divisible_by_100_but_not_400 PASSED                                        [ 20%]
ass_8_8_2.py::test_divisible_by_4_but_not_100 PASSED                                          [ 30%]
ass_8_8_2.py::test_not_divisible_by_4 PASSED                                                  [ 40%]
ass_8_8_2.py::test_edge_case_year_0 PASSED                                                    [ 50%]
ass_8_8_2.py::test_negative_years PASSED                                                      [ 60%]
ass_8_8_2.py::test_recent_years PASSED                                                        [ 70%]
ass_8_8_2.py::test_century_boundaries PASSED                                                  [ 80%]
ass_8_8_2.py::test_future_years FAILED                                                        [ 90%]
ass_8_8_2.py::test_invalid_inputs PASSED                                                      [100%]

============================================== FAILURES ==============================================
_____________________________________________ test_future_years _____________________________________________

    def test_future_years():
        assert is_leap_year(3000) == False
>       assert is_leap_year(3200) == False
E       assert True == False
E        +  where True = is_leap_year(3200)

ass_8_8_2.py:42: AssertionError
======================================= short test summary info =======================================
FAILED ass_8_8_2.py::test_future_years - assert True == False
========================================= 1 failed, 9 passed in 0.13s =========================================
PS C:\Users\Himes\Documents\.Dev\Python>
```

9. Write a function SumOfDigits and validate its implementation using 7 pytest test cases.

```python
def sum_of_digits(n):
    if not isinstance(n, (int, float)):
        return 0
    if isinstance(n, float):
        n = int(n)
    n = abs(n)
    total = 0
    while n > 0:
        total += n % 10
        n //= 10
    return total
```

```python
from ass_8_9_1 import sum_of_digits

def test_positive_numbers():
    assert sum_of_digits(123) == 6
    assert sum_of_digits(456) == 15
    assert sum_of_digits(9876) == 30

def test_single_digit():
    assert sum_of_digits(5) == 5
    assert sum_of_digits(0) == 0
    assert sum_of_digits(9) == 9

def test_negative_numbers():
    assert sum_of_digits(-123) == 6
    assert sum_of_digits(-5) == 5
    assert sum_of_digits(-9876) == 30

def test_large_numbers():
    assert sum_of_digits(999999) == 54
    assert sum_of_digits(1000000) == 1
    assert sum_of_digits(123456789) == 45

def test_numbers_with_zeros():
    assert sum_of_digits(101) == 2
    assert sum_of_digits(1001) == 2
    assert sum_of_digits(10001) == 2

def test_float_inputs():
    assert sum_of_digits(123.45) == 6
    assert sum_of_digits(45.67) == 9
    assert sum_of_digits(0.5) == 0

def test_invalid_inputs():
    assert sum_of_digits("123") == 0
    assert sum_of_digits(None) == 0
    assert sum_of_digits([123]) == 0
```

Output:

```
PS C:\Users\Himes\Documents\.Dev\Python> pytest ass_8_9_2.py -v
========================================== test session starts ==========================================
platform win32 -- Python 3.14.0, pytest-9.0.2, pluggy-1.6.0 -- C:\Users\Himes\AppData\Local\Programs\Python\Python314\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\Himes\Documents\.Dev\Python
collected 7 items

ass_8_9_2.py::test_positive_numbers PASSED                                                    [ 14%]
ass_8_9_2.py::test_single_digit PASSED                                                        [ 28%]
ass_8_9_2.py::test_negative_numbers PASSED                                                    [ 42%]
ass_8_9_2.py::test_large_numbers PASSED                                                       [ 57%]
ass_8_9_2.py::test_numbers_with_zeros PASSED                                                  [ 71%]
ass_8_9_2.py::test_float_inputs PASSED                                                        [ 85%]
ass_8_9_2.py::test_invalid_inputs PASSED                                                      [100%]

========================================== 7 passed in 0.04s ==========================================
PS C:\Users\Himes\Documents\.Dev\Python>
```

10. Write a function SortNumbers (implement bubble sort) and validate its implementation using 25 pytest test cases.

```python
def bubble_sort(arr):
    if not isinstance(arr, list):
        return []
    if not arr:
        return []

    result = arr.copy()
    n = len(result)

    for i in range(n):
        for j in range(0, n-i-1):
            if result[j] > result[j+1]:
                result[j], result[j+1] = result[j+1], result[j]
    return result
```

```python
from ass_8_10_1 import bubble_sort

def test_empty_list():
    assert bubble_sort([]) == []

def test_single_element():
    assert bubble_sort([5]) == [5]
    assert bubble_sort([-3]) == [-3]
```

```python
    assert bubble_sort([0]) == [0]

def test_two_elements_sorted():
    assert bubble_sort([1, 2]) == [1, 2]

def test_two_elements_unsorted():
    assert bubble_sort([2, 1]) == [1, 2]

def test_three_elements_sorted():
    assert bubble_sort([1, 2, 3]) == [1, 2, 3]

def test_three_elements_unsorted():
    assert bubble_sort([3, 1, 2]) == [1, 2, 3]
    assert bubble_sort([2, 3, 1]) == [1, 2, 3]

def test_negative_numbers():
    assert bubble_sort([-5, -2, -8, -1]) == [-8, -5, -2, -1]

def test_mixed_positive_negative():
    assert bubble_sort([3, -1, 0, 5, -7]) == [-7, -1, 0, 3, 5]

def test_duplicate_numbers():
    assert bubble_sort([3, 1, 3, 2, 1]) == [1, 1, 2, 3, 3]

def test_already_sorted():
    assert bubble_sort([1, 2, 3, 4, 5]) == [1, 2, 3, 4, 5]

def test_reverse_sorted():
    assert bubble_sort([5, 4, 3, 2, 1]) == [1, 2, 3, 4, 5]

def test_all_same():
    assert bubble_sort([5, 5, 5, 5]) == [5, 5, 5, 5]

def test_large_numbers():
    assert bubble_sort([1000, 500, 2000, 1500]) == [500, 1000, 1500, 2000]

def test_zero_and_negatives():
    assert bubble_sort([0, -1, -5, 0, 3]) == [-5, -1, 0, 0, 3]

def test_alternating_pattern():
    assert bubble_sort([1, 3, 2, 4, 3, 5]) == [1, 2, 3, 3, 4, 5]

def test_ten_elements():
```

```
    assert bubble_sort([10, 9, 8, 7, 6, 5, 4, 3, 2, 1]) == [1, 2, 3, 4, 5,
6, 7, 8, 9, 10]

def test_float_numbers():
    assert bubble_sort([3.5, 1.2, 2.8, 1.5]) == [1.2, 1.5, 2.8, 3.5]

def test_mixed_int_float():
    assert bubble_sort([3, 1.5, 2, 0.5]) == [0.5, 1.5, 2, 3]

def test_small_range():
    assert bubble_sort([-100, 100, 0, -50, 50]) == [-100, -50, 0, 50, 100]

def test_consecutive_numbers():
    assert bubble_sort([5, 4, 3, 2, 1, 0]) == [0, 1, 2, 3, 4, 5]

def test_odd_number_of_elements():
    assert bubble_sort([7, 3, 9, 1, 5]) == [1, 3, 5, 7, 9]

def test_even_number_of_elements():
    assert bubble_sort([8, 4, 6, 2]) == [2, 4, 6, 8]

def test_list_with_extremes():
    assert bubble_sort([-999, 999, 0, -1, 1]) == [-999, -1, 0, 1, 999]

def test_non_list_input():
    assert bubble_sort("not a list") == []
    assert bubble_sort(123) == []
    assert bubble_sort(None) == []

def test_preserves_original():
    original = [3, 1, 2]
    bubble_sort(original)
    assert original == [3, 1, 2]
```

Output:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS                                                                    powershell

PS C:\Users\Himes\Documents\.Dev\Python> pytest ass_8_10_2.py -v
========================================================================================== test session starts ==========================================================================================
platform win32 -- Python 3.14.0, pytest-9.0.2, pluggy-1.6.0 -- C:\Users\Himes\AppData\Local\Programs\Python\Python314\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\Himes\Documents\.Dev\Python
collected 25 items

ass_8_10_2.py::test_empty_list PASSED                                                                                                                                                                  [  4%]
ass_8_10_2.py::test_single_element PASSED                                                                                                                                                              [  8%]
ass_8_10_2.py::test_two_elements_sorted PASSED                                                                                                                                                         [ 12%]
ass_8_10_2.py::test_two_elements_unsorted PASSED                                                                                                                                                       [ 16%]
ass_8_10_2.py::test_three_elements_sorted PASSED                                                                                                                                                       [ 20%]
ass_8_10_2.py::test_three_elements_unsorted PASSED                                                                                                                                                     [ 24%]
ass_8_10_2.py::test_negative_numbers PASSED                                                                                                                                                            [ 28%]
ass_8_10_2.py::test_mixed_positive_negative PASSED                                                                                                                                                     [ 32%]
ass_8_10_2.py::test_duplicate_numbers PASSED                                                                                                                                                           [ 36%]
ass_8_10_2.py::test_already_sorted PASSED                                                                                                                                                              [ 40%]
ass_8_10_2.py::test_reverse_sorted PASSED                                                                                                                                                              [ 44%]
ass_8_10_2.py::test_all_same PASSED                                                                                                                                                                    [ 48%]
ass_8_10_2.py::test_large_numbers PASSED                                                                                                                                                               [ 52%]
ass_8_10_2.py::test_zero_and_negatives PASSED                                                                                                                                                          [ 56%]
ass_8_10_2.py::test_alternating_pattern PASSED                                                                                                                                                         [ 60%]
ass_8_10_2.py::test_ten_elements PASSED                                                                                                                                                                [ 64%]
ass_8_10_2.py::test_float_numbers PASSED                                                                                                                                                               [ 68%]
ass_8_10_2.py::test_mixed_int_float PASSED                                                                                                                                                             [ 72%]
ass_8_10_2.py::test_small_range PASSED                                                                                                                                                                 [ 76%]
ass_8_10_2.py::test_consecutive_numbers PASSED                                                                                                                                                         [ 80%]
ass_8_10_2.py::test_odd_number_of_elements PASSED                                                                                                                                                      [ 84%]
ass_8_10_2.py::test_even_number_of_elements PASSED                                                                                                                                                     [ 88%]
ass_8_10_2.py::test_list_with_extremes PASSED                                                                                                                                                          [ 92%]
ass_8_10_2.py::test_non_list_input PASSED                                                                                                                                                              [ 96%]
ass_8_10_2.py::test_preserves_original PASSED                                                                                                                                                          [100%]

========================================================================================== 25 passed in 0.08s ==========================================================================================
PS C:\Users\Himes\Documents\.Dev\Python>
```

11. Write a function ReverseString and validate its implementation using 5 unittest test cases.

12. Write a function AnagramChecker and validate its implementation using 10 unittest test cases.

13. Write a function ArmstrongChecker and validate its implementation using 8 unittest test cases